

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

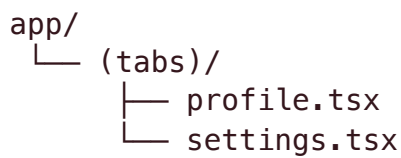
Last Week's Lecture

- **Navigation with React Navigation**
 - Dynamic configurations
 - Parameter passing (`route` or custom props)
- **Navigation with Expo Router**
 - Navigation: `router.push()`
 - Dynamic routes: `[id]`
 - Access params with `useLocalSearchParams()`

Route Group

Parentheses () in folder names = route group

- Group files together without adding extra path segments in the URL
- Organize screens without changing their route paths



```
app/  
└─ (tabs)/  
    └─ profile.tsx  
    └─ settings.tsx
```

- URL `/profile` → `app/(tab)/profile.tsx`
- URL `/settings` → `app/(tab)/settings.tsx`
- `(tabs)` is not part of the URL

Route Group: Example

| app/(tabs)/profile.tsx

```
1 import { View, Text } from "react-native";
2 import { router } from "expo-router";
3 import { globalStyles } from "../../styles/globalStyles";
4 import PrimaryButton from "../../components/PrimaryButton";
5
6 export default function Profile() {
7   return (
8     <View style={globalStyles.container}>
9       <Text style={globalStyles.headerText}>Profile Screen</Text>
10      <PrimaryButton onPress={() => router.push("/")}>Go to Home</Primary
11    </View>
12  );
13 }
```

Route Group: Example

Navigate to `app/(tab)/profile.tsx` in `app/index.tsx`

| `app/index.tsx`

```
1 import { View, Text } from "react-native";
2 import { router } from "expo-router";
3 import { globalStyles } from "../styles/globalStyles";
4 import PrimaryButton from "../components/PrimaryButton";
5
6 export default function Home() {
7   return (
8     <View style={globalStyles.container}>
9       <Text style={globalStyles.headerText}>Home Screen</Text>
10      <PrimaryButton onPress={() => router.push("/profile")}>
11        Go to Profile
12      </PrimaryButton>
13    </View>
14  );
15 }
```

Duplicate Routes

What happens if I also have `app/profile.tsx`?

`app/profile.tsx`

```
1 import { View, Text } from "react-native";
2 import { router } from "expo-router";
3 import { globalStyles } from "../styles/globalStyles";
4 import PrimaryButton from "../components/PrimaryButton";
5
6 export default function Profile() {
7   return (
8     <View style={globalStyles.container}>
9       <Text style={globalStyles.headerText}>app/profile Screen</Text>
10      <PrimaryButton onPress={() => router.push("/")}>Go to Home</Primary
11    </View>
12  );
13 }
```

Duplicate Routes

What happens if I also have `app/profile.tsx`?

`router.push("/profile")` → navigates to
`app/profile.tsx`

- Because that's the top-level route for `/profile`

If multiple files match the same path, the one outside route groups wins

Why Use Route Groups?

Route groups are for **organization and layout sharing**, *not* for changing URLs

- Group related screens under the same layout (e.g. tab navigation)
- Avoid adding (`tabs`) or (`auth`) to the actual URL
- Use different layouts for different sections of the app

When to Use Route Group

- Organize related screens
- Share layouts
- Example: All (`tabs`) screens show the tab bar automatically
- Hide folder name from URL
- Example: (`auth`) not included in route path for clean URLs while maintaining folder structure

Using route groups to create different versions of the same route (`/profile`) can work, but causes confusion

index.tsx

Define the default screen for a directory

```
app/  
├─ index.tsx      # root-level default screen → "/"  
└─ details/  
    └─ index.tsx  # default screen for "/details"
```

- `index.tsx` automatically maps to the folder's URL path
- `router.push("/details")` → `app/details/index.tsx`
- No need to name it after the route

Make file-based routing clean and intuitive

`_layout.tsx`

Define a layout or navigator for all screens in a folder

```
app/
├── (tabs)
│   ├── _layout.tsx    # Tab navigator layout
│   ├── home.tsx
│   └── profile.tsx
└── index.tsx
```

- `_layout.tsx` wraps all child screens in the folder
- Can define Stack/Tabs/Drawer navigators, or custom layout
- At the root folder, `app/_layout.tsx` replaces `App.tsx` as the app entry point

`_layout.tsx`: Example

```
app/
├── (tabs)
│   ├── _layout.tsx    # Tab navigator layout
│   ├── home.tsx
│   └── profile.tsx
└── index.tsx
```

`app/(tabs)/_layout.tsx`

```
1 import { Tabs } from "expo-router";
2
3 export default function TabsLayout() {
4   return <Tabs></Tabs>;
5 }
```

Navigation Pattern

- Stack: docs.expo.dev/router/advanced/stack/
- Tab: docs.expo.dev/router/advanced/tabs/
- Drawer: docs.expo.dev/router/advanced/drawer/
- Modal: docs.expo.dev/router/advanced/modals/
- Present content on top of the existing screen

Recap: Expo Router

- **File-Based Routing:** Define routes via `app/` directory
 - `index.tsx`: `"/`
 - `_layout.tsx`: Layout / navigator of all screens in a folder
 - `[id].tsx`: Dynamic routes
 - `(tabs)/`: Route group
- **Navigation:** `router.push("/path")`
- Access params with `useLocalSearchParams()`
- **Type Safety:** Auto-infer types from files

vs. React Navigation

Feature	React Navigation	Expo Router
Routing Style	Manual config (e.g., <code><Stack.Navigator></code>)	File-based (<code>app/[id].tsx</code>)
Setup Complexity	Explicit screens, params types	Auto-routes, inferred types
Navigation API	<code>navigate("Name", params)</code>	<code>router.push("/path")</code>
Params	<code>route.params</code>	<code>useLocalSearchParams</code>
Type Safety	Manual <code>ParamList</code> definitions	Automatic from file structure

Which to Choose

Expo Router: Expo ecosystem + File-driven workflows

- Simple/intuitive apps
- Auto-types
- Web support

React Navigation

- Complex/custom navigation (e.g., non-file logic)
- Full control needed
- Non-Expo projects

Recap: Expo Router

Expo Router: File-based routing

- Navigation: `router.push()`
- Dynamic routes: `[id]`
- Route groups: `(tabs)`

Key Trade-Offs

- **React Navigation:** Maximum flexibility & control
- **Expo Router:** Fast setup, intuitive, especially in Expo apps

Today's Lecture

State Management

Assignment 3: Navigation with Expo Router and State Management

Assignment 4: Advanced State Management and Persistence

State Management: Why?

Assignment 2 passes state (activities array) via props

```
<App>  
  <HomeScreen activities={activities} />  
    <ActivityListItem activities={activities} />
```

- **Prop Drilling:** Passing the same data through many layers of components
- **Scalability Issue:** Adding screens or features → Need to keep passing the same props everywhere

State Management: Goals

- Share state across components without passing props
- Keep code clean, readable, and maintainable
- Make it easier to add new screens or features

State Management: Tools

- **Context API**: Global state without prop drilling
- **useReducer**: Managing complex local state
- **Redux Toolkit**: Scalable global state for larger apps

Local vs. Global State

Local State (`useState` / `useReducer`)

- Stored **inside a single component**
- Example: text input, form values
- Easy to manage, no extra libraries needed

Global State

- Shared **across multiple components**
- Example: activities list, logged-in user info
- Require state management solution (Context API, Redux Toolkit, etc.)

When to Use Global State

- Data used by multiple unrelated components
- Passing props through many levels frequently
- App grows beyond 3-4 screens and state needs to be shared

Context API

React's built-in solution for sharing state across components

- **No external libraries:** Lightweight and always available in React/React Native
- **Wrap parts of your app:** Components inside this wrapper can access shared state directly
- **Ideal for:** Medium-sized apps or simple global data

React Doc: Passing Data Deeply with Context

Context API: Basics

1. Create Context

```
const MyContext = createContext(defaultValue)
```

- Define a shared “container” for state
- Set an optional default value

2. Provider

```
<MyContext.Provider value={someValue}>  
  <ChildComponent />  
</MyContext.Provider>
```

- Wrap components that should access the shared state
- Pass the current value to all its descendants

Context API: Basics

- 1. Create Context
- 2. Provider
- 3. Consumer
- Use `useContext(MyContext)` to read or use the shared value inside any child component

Context API: Example

Goal: Share a simple state (theme) across components without prop drilling

App Features

- Display current theme (light / dark)
- Have a button to toggle theme

```
<Text>Current theme: light</Text>  
<Button title="Toggle Theme" />
```

- Make theme shared across screens

The “Old” Way

app/index.tsx

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useState } from "react";
3
4 export default function HomeScreen() {
5   const [theme, setTheme] = useState("light");
6
7   const toggleTheme = () => {
8     setTheme((prev) => (prev === "light" ? "dark" : "light"));
9   };
10
11   return (
12     <View
13       style={styles.container, theme === "light" ? styles.light : styles
14     >
15       <Text style={styles.text}>Current theme: {theme}</Text>
16       <Button title="Toggle Theme" onPress={toggleTheme} />
17     </View>
18   );
19 }
```

- State is local → only this screen knows about `theme`
- If another screen needs `theme` → need to pass props → prop drilling

Refactor with Context API

Goal: Move `theme` to a shared global context

Steps

1. Create a new context (`ThemeContext`)
2. Wrap the app in `ThemeProvider`
3. Use `useContext` to access and update theme anywhere

Create a Context

Create `contexts/ThemeContext.tsx`

```
context-api-demo/contexts/ThemeContext.tsx
```

```
import { createContext } from "react";

const ThemeContext = createContext({
  theme: "light",
  toggleTheme: () => {},
});
```

- Create a “container” to hold shared data

Add Type Safety

Define a type for the context's data shape

types.ts

```
1 export type ThemeContextType = {  
2   theme: "light" | "dark";  
3   toggleTheme: () => void;  
4 };
```

Create a typed context

context-api-demo/contexts/ThemeContext.tsx

```
1 import { createContext } from "react";  
2 import { ThemeContextType } from "../types";  
3  
4 export const ThemeContext = createContext<ThemeContextType>({  
5   theme: "light",  
6   toggleTheme: () => {},  
7 });
```

Define Context

context-api-demo/contexts/ThemeContext.tsx

```
export const ThemeContext = createContext<ThemeContextType>({  
  theme: "light",  
  toggleTheme: () => {},  
});
```

- **The default value is fake:** Never actually used once the real provider wraps the app
- If someone forgets to wrap with `<ThemeProvider>`, components might still run with this placeholder, silently causing bugs

Safe Version

Let TypeScript warn if Provider is missing

```
context-api-demo/contexts/ThemeContext.tsx
```

```
export const ThemeContext = createContext<ThemeContextType | undefined>(undef
```

Use with a helper hook

```
context-api-demo/contexts/ThemeContext.tsx
```

```
export function useTheme(): ThemeContextType {  
  const context = useContext(ThemeContext);  
  if (!context) {  
    throw new Error("useTheme must be used within a ThemeProvider");  
  }  
  return context;  
}
```

Add a Provider

Make a Provider component that stores the current theme state and provides it to all child components

context-api-demo/contexts/ThemeContext.tsx

```
1 import { createContext, useState, ReactNode } from "react";
2 import { ThemeContextType } from "../types";
3
4 const ThemeContext = createContext<ThemeContextType | undefined>(undefined);
5
6 export function ThemeProvider({ children }: { children: ReactNode }) {
7   const [theme, setTheme] = useState<"light" | "dark">("light");
8
9   const toggleTheme = () => {
10     setTheme((prev) => (prev === "light" ? "dark" : "light"));
11   };
12
13   return (
14     <ThemeContext.Provider value={{ theme, toggleTheme }}>
15       {children}
16     </ThemeContext.Provider>
17   );
18 }
```

ThemeProvider

| context-api-demo/contexts/ThemeContext.tsx

```
export function ThemeProvider({ children }: { children: ReactNode }) {  
  const [theme, setTheme] = useState<"light" | "dark">("light");  
  const toggleTheme = () => {  
    setTheme((prev) => (prev === "light" ? "dark" : "light"));  
  };  
  return (  
    <ThemeContext.Provider value={{ theme, toggleTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}
```

- **ThemeProvider** manages the data (**theme**, **toggleTheme**)
- **<ThemeContext.Provider>** shares the data with all nested components
- Every component inside can “subscribe” to updates

ThemeContext.tsx

context-api-demo/contexts/ThemeContext.tsx

```
1 import { createContext, useContext, useState, ReactNode } from "react";
2 import { ThemeContextType } from "../types";
3
4 const ThemeContext = createContext<ThemeContextType | undefined>(undefined);
5
6 export function ThemeProvider({ children }: { children: ReactNode }) {
7   const [theme, setTheme] = useState<"light" | "dark">("light");
8
9   const toggleTheme = () => {
10     setTheme((prev) => (prev === "light" ? "dark" : "light"));
11   };
12
13   return (
14     <ThemeContext.Provider value={{ theme, toggleTheme }}>
15       {children}
16     </ThemeContext.Provider>
17   );
18 }
19
```

Wrap the App

app/_layout.tsx

```
1 import { Stack } from "expo-router";
2 import { ThemeProvider } from "../contexts/ThemeContext";
3
4 export default function RootLayout() {
5   return (
6     <ThemeProvider>
7       <Stack />
8     </ThemeProvider>
9   );
10 }
```

Every screen inside `<ThemeProvider>` can use the same theme state

- No need to pass `theme` or `toggleTheme` through props anymore

Consume Context

Now we can use the context anywhere

- For example, in [HomeScreen](#)

Live Coding

app/index.tsx

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useState } from "react";
3
4 export default function HomeScreen() {
5   const [theme, setTheme] = useState("light");
6
7   const toggleTheme = () => {
8     setTheme((prev) => (prev === "light" ? "dark" : "light"));
9   };
10
11   return (
12     <View
13       style={[styles.container, theme === "light" ? styles.light : styles
14     >
15       <Text style={styles.text}>Current theme: {theme}</Text>
16       <Button title="Toggle Theme" onPress={toggleTheme} />
17     </View>
18   );
19 }
```

Consume Context

Use context in `HomeScreen`

`app/index.tsx`

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useTheme } from "../contexts/ThemeContext";
3
4 export default function HomeScreen() {
5   const { theme, toggleTheme } = useTheme();
6
7   return (
8     <View
9       style={[styles.container, theme === "light" ? styles.light : styles
10     >
11       <Text style={styles.text}>Current theme: {theme}</Text>
12       <Button title="Toggle Theme" onPress={toggleTheme} />
13     </View>
14   );
15 }
16
17 const styles = StyleSheet.create({
18   container: { flex: 1, justifyContent: "center", alignItems: "center" },
19   light: { backgroundColor: "#fff" },
```

`useTheme()` gives direct access to shared data (`theme`) and functions (`toggleTheme`) — no props, no drilling

Recap: Context API

React's built-in solution for sharing state

. Create a Context

- Define shared context (`theme`, `toggleTheme`)
- Add a type `ThemeContextType` for clarity and safety

?. Provide the Context

- Wrap the app with `<ThemeProvider>`
- Make shared state accessible to all screens

}. Consume the Context

- Use `useTheme()` hook to read and update global data

Assignment 3

Implement global state management using Context API with TypeScript

- Create a `contexts/ActivityContext.tsx` file to manage `activities` array globally
- Implement `ActivityProvider` that uses `useState` to manage `activities`
- Provide functions (`addActivity`, `updateActivity`, `deleteActivity`) to modify the state
- Wrap the app in `ActivityProvider` in `app/_layout.tsx`

Context API: Limitation

Context API works well for simple global state (e.g., theme, user settings), but can become challenging as app grows:

- All consumers re-render on any change → May affect performance
- Not ideal for complex state logic → Better to use `useReducer` or Redux

useReducer

A React hook for managing complex or related local state

Official Doc: [useReducer](#)

Key idea: Instead of multiple `useState` calls, `useReducer` lets you define all state updates in one place (the **reducer** function)

Why use `useReducer`?

- `useState` works well for **simple state**
- For **complex state updates** or **multiple related state values**, `useReducer` helps

Advantages:

- **Centralize state logic** in a single reducer function
- **Predictable updates:** Given an action, the state changes in a defined way
- Can integrate with **Context API** for global state

useReducer: Basic Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- `state`: Current state
- `dispatch`: Function to trigger state changes
- `reducer`: Function `(state, action) => newState`

Example: Simple Counter

Build a small counter app to demonstrate `useReducer`

- Show the current count in a `<Text>` component
- Provide two buttons:
 - “+” button: Increment the count
 - “-” button: Decrement the count

Example: Simple Counter

usereducer-demo/App.tsx

```
1 import { useReducer } from "react";
2 import { View, Text, Button } from "react-native";
3
4 type State = { count: number };
5 type Action = { type: "increment" | "decrement" };
6
7 function reducer(state: State, action: Action): State {
8   switch (action.type) {
9     case "increment":
10       return { count: state.count + 1 };
11     case "decrement":
12       return { count: state.count - 1 };
13     default:
14       return state;
15   }
16 }
17
18 export default function Counter() {
19   const [state, dispatch] = useReducer(reducer, { count: 0 });
```

reducer defines all state update rules

dispatch triggers changes based on action type

Live Demo

usereducer-demo/App.tsx

```
1 import { useReducer } from "react";
2 import { View, Text, Button, StyleSheet } from "react-native";
3
4 type State = { count: number };
5 type Action = { type: "increment" | "decrement" };
6
7 function reducer(state: State, action: Action): State {
8   switch (action.type) {
9     case "increment":
10       return { count: state.count + 1 };
11     case "decrement":
12       return { count: state.count - 1 };
13     default:
14       return state;
15   }
16 }
17
18 export default function Counter() {
19   const [state, dispatch] = useReducer(reducer, { count: 0 });
```

reducer defines all state update rules

dispatch triggers changes based on action type

Work nicely with Context API for global state management

Combine w/ Context API

Use `useReducer` with Context API

1. Create a reducer function for your state logic
2. Wrap your app with a Provider (e.g., `ThemeProvider`)
3. Pass `[state, dispatch]` as the context value
4. Consume in any child component using `useContext`

```
// pseudo-code
<CounterContext.Provider value={{ state, dispatch }}>
  <AppScreens />
</CounterContext.Provider>
```

Combine w/ Context API

`useReducer` handles complex state logic

Context API makes it accessible globally

Recap: `useReducer`

A React hook for managing complex or related local state

- Provide predictable state updates
- Centralize all state logic in a single reducer function
- Good for **complex or related local state**
- Can be combined with Context API for shared/global state

Key takeaway: Use `useReducer` when `useState` becomes messy or your state updates need clear, structured logic

Limitations

Context API

- All consumers re-render on any state change → may hurt performance
- Not ideal for very complex state logic across large apps
- `useReducer`
- Handle complex state locally
- Scaling issues: Managing state across many screens still requires prop drilling or wrapping many providers

State Management

For **medium apps**, Context + useReducer works fine

For **large or complex apps**, a more structured state management solution is better

- Redux Toolkit

Course Project Update

Added Redux Toolkit as an option for state management

Redux Toolkit

Industry-standard library for managing global state in React/React Native

Note: Redux Toolkit (RTK) is built on top of **Redux**

- Not Redux itself, but a simplified, modern way to use Redux

Official Doc: Redux Toolkit

Redux Toolkit

Industry-standard library for managing global state in React/React Native

- Provide predictable state updates with centralized logic
- Reduce boilerplate compared to classic Redux
- No need to write action types, action creators, or switch statements manually
- Integrate easily with React components via hooks
- Compatible with Redux DevTools and other Redux ecosystem tools

Key Concepts

- **Slice:** A “chunk” of state with its logic and actions bundled together
- Example: Theme slice → stores current theme + defines how to toggle it
- **Store:** Centralized state container
- **Provider:** Makes store accessible to all components
- **Hooks:** Functions to interact with store
 - `useSelector`: Read state from the store
 - `useDispatch`: Trigger actions to update state

Setup

Install the required packages for Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
```

- `@reduxjs/toolkit`: Provide helper functions to define slices, reducers, and store easily
- `react-redux`: Provide the Provider component and hooks (`useSelector`, `useDispatch`) to connect React/React Native components to the store

Example: Theme Switcher

Mirror the Context API theme example

- Show current theme (light/dark)
- Toggle theme using Redux Toolkit

Create a Slice

Define state + reducer logic + actions in one place

redux-demo/features/theme/themeSlice.ts

```
1 import { createSlice } from "@reduxjs/toolkit";
2
3 type ThemeState = { theme: "light" | "dark" };
4
5 const initialState: ThemeState = { theme: "light" };
6
7 const themeSlice = createSlice({
8   name: "theme",
9   initialState,
10  reducers: {
11    toggleTheme: (state) => {
12      state.theme === "light" ? "dark" : "light";
13    },
14  },
15 });
16
17 // Action creators are generated for each case reducer function
18 export const { toggleTheme } = themeSlice.actions;
19
```

Configure Store

Add slice reducers to a centralized Redux store

redux-demo/store/store.ts

```
1 import { configureStore } from "@reduxjs/toolkit";
2 // Default export can be imported with any name
3 import themeReducer from "../features/theme/themeSlice";
4
5 export const store = configureStore({
6   reducer: {
7     theme: themeReducer, // key = slice name, value = reducer
8   },
9 });
10
11 // Infer the `RootState` and `AppDispatch` types from the store
12 export type RootState = ReturnType<typeof store.getState>;
13 export type AppDispatch = typeof store.dispatch;
```

- Use `RootState` and `AppDispatch` for TypeScript safety
- Ready to wrap app with `<Provider store={store}>`

Wrap App with Provider

Make store accessible to all screens

redux-demo/app/_layout.tsx

```
1 import { Stack } from "expo-router";
2 import { Provider } from "react-redux";
3 import { store } from "../store/store";
4
5 export default function RootLayout() {
6   return (
7     <Provider store={store}>
8       <Stack />
9     </Provider>
10  );
11 }
```

Use State & Dispatch

- `useSelector`: Read state from the Redux store
- `useDispatch`: Trigger actions to update state

```
const theme = useSelector((state: RootState) => state.theme.theme);  
const dispatch: AppDispatch = useDispatch();  
  
dispatch(toggleTheme());
```

Conceptual parallel

- **Context API**: Provider + `useContext`
- **Redux Toolkit**: Store + Provider + `useSelector` / `useDispatch`

Use State & Dispatch

useSelector: Read state from the Redux store

useDispatch: Trigger actions to update state

redux-demo/app/index.tsx

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useSelector, useDispatch } from "react-redux";
3 import { RootState, AppDispatch } from "../store/store";
4 import { toggleTheme } from "../features/theme/themeSlice";
5
6 export default function HomeScreen() {
7   const theme = useSelector((state: RootState) => state.theme.theme);
8   const dispatch: AppDispatch = useDispatch();
9
10  return (
11    <View
12      style={[styles.container, theme === "light" ? styles.light : styles
13    >
14      <Text style={styles.text}>Current theme: {theme}</Text>
15      <Button title="Toggle Theme" onPress={() => dispatch(toggleTheme())
16    </View>
17  );
18 }
19
```

Live Demo

Nothing Change

To be fixed in next lecture

Speaker notes